Single Linked List

Tujuan Pembelajaran

Setelah mempelajari modul ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar dan struktur dari Single Linked List.
2. Mendeklarasikan simpul (*node*) dan *pointer* di dalam bahasa pemrograman C++.
3. Memahami berbagai operasi dasar pada Single Linked List (*transversal, insert, delete, update*).
4. Mengimplementasikan operasi-operasi Single Linked List tersebut ke dalam program sederhana.

Apa Itu Linked List?

Linked list merupakan salah satu jenis struktur data yang terdiri dari kumpulan data yang tersusun secara berurutan, saling terhubung, bersifat dinamis, dan terbatas. Setiap elemen di dalam linked list, disebut simpul (node), saling terkait dalam urutan tertentu. Simpul ini biasanya didefinisikan sebagai sebuah struktur atau class yang menyimpan data. Dalam konsepnya, linked list terdiri dari sejumlah node yang dihubungkan secara linear menggunakan pointer.

Deklarasi Node

Pada linked list, setiap simpul (node) biasanya terdiri dari dua bagian utama: bagian pertama menyimpan data, dan bagian kedua menyimpan referensi atau pointer yang menghubungkan simpul tersebut ke simpul berikutnya. Di dalam bahasa pemrograman seperti C++, simpul ini dapat dideklarasikan menggunakan struct atau class. Kita bisa deklarasi node langsung atau pakai nested struct.

```
#include <iostream>
using namespace std;
struct mahasiswa{
   string nama;
   int nim;
};
struct node{
  mahasiswa mhs;
  node*next;
};
struct node*head=NULL;
struct node*last=NULL;
```

```
#include <iostream>
using namespace std;

struct Node {
  int data;
  Node *next;
};

int main(){
  Node*head=nullptr;
  return 0;
}
```

struct Node : Kita membuat node menggunakan struct, yaitu keyword yang berfungsi untuk mengelompokan kumpulan variabel ke dalam 1 nama. Di atas kita mengelompokan variabelnya kedalamstruct yang bernama Node.

Node *next : artinya kita membuat variabel pointer yang bisa menyimpan alamat yang bertipe data struct Node.

Node *HEAD :Berfungsi untuk menyimpan alamat dari Node pertama, saat deklarasi kita berikan nilai

NULL/nullptr/0 karena linked list dianggap masih kosong.

Operasi Pada Linked List

Operasi pada Linked List mencakup berbagai manipulasi dasar yang dapat dilakukan pada struktur data ini, seperti menambah, menghapus, dan mencari elemen. Tidak seperti array, Linked List memungkinkan pengelolaan data yang lebih fleksibel karena elemen-elemen tidak perlu disimpan secara berurutan dalam memori. Operasi-operasi ini biasanya melibatkan penyesuaian referensi atau "pointer" yang menghubungkan node-node di dalam Linked List. Berikut adalah operasi-operasi yang ada pada linked list :

Find Last

Fungsi ini mencari elemen terakhir dari linked list menggunakan variabel tail.

isEmpty

Fungsi ini menentukan apakah linked list kosong atau tidak.

```
bool isEmpty(){
  return head==NULL;
}
```

Insert

Istilah Insert berarti menambahkan sebuah simpul baru ke dalam suatu linked list.

Retrieve

Fungsi ini mengambil elemen yang ditunjuk. Elemen tersebut lalu dikembalikan oleh fungsi.

Update

Fungsi ini mengubah isi dari elemen yang ditunjuk.

Delete Head

Fungsi ini menghapus elemen yang ditunjuk head. Head berpindah ke elemen sesudahnya.

Delete Tail

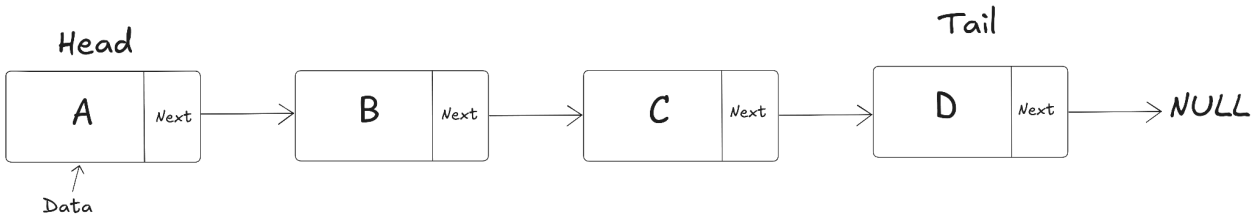
Fungsi ini menghapus elemen yang ditunjuk tail. Tail berpindah ke elemen sebelumnya.

Clear

Fungsi ini menghapus linked list yang sudah ada.

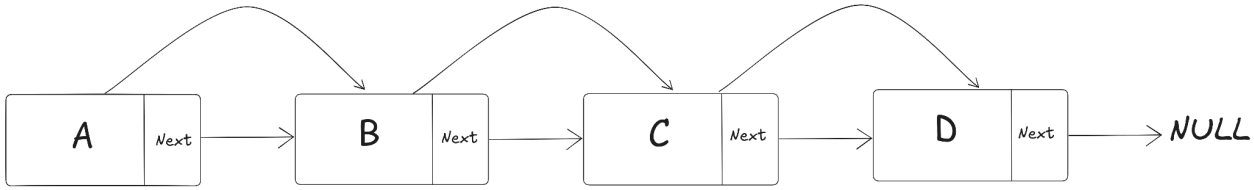
```
void clearList(Node *&head) {
    Node *temp;
    while (head != NULL) {
        temp = head; // simpan pointer sementara
        head = head->next; // pindah ke node berikutnya
        delete temp; // hapus node lama
    }
    cout << "Linked list berhasil dikosongkan." << endl;
}
```

Single Linked List



Single linked list merupakan jenis linked list yang paling simple, karena setiap node hanya memiliki satu pointer yang mengarah ke node berikutnya. Dengan demikian, transversing data hanya bisa dilakukan searah, yaitu dari node pertama (head) menuju node terakhir (tail) yang menunjuk ke NULL, terdapat beberapa operasi yang dapat kita lakukan dalam single linked list, yakni :

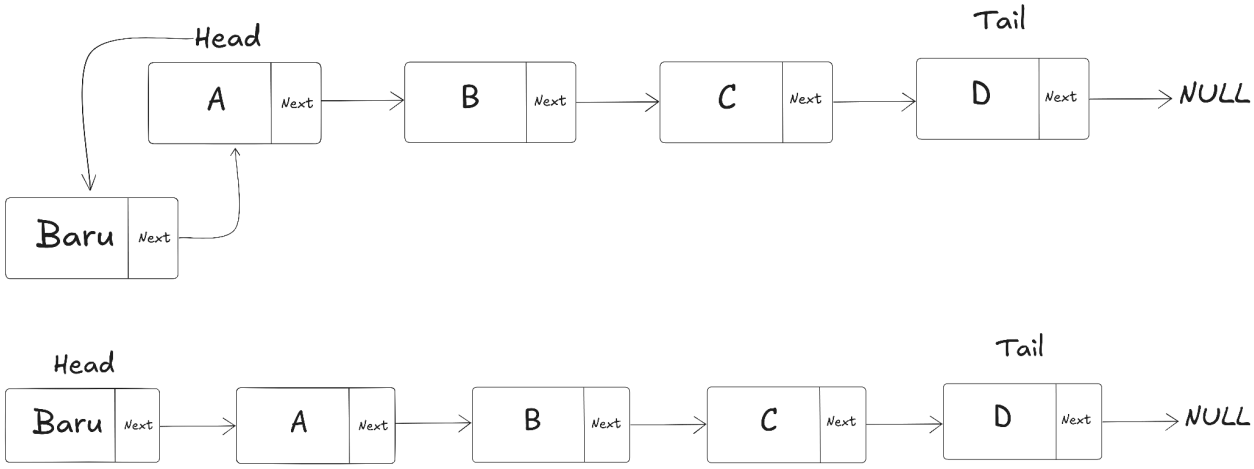
Transversal



Traversal adalah proses mengunjungi setiap elemen Node secara berurutan, dimulai dari node pertama. Karena linked list tidak memiliki fitur indexing data, kita harus melakukan traversal untuk mencapai Node yang ingin diakses.

```
// Fungsi transversal (menampilkan isi list)
void transversal(Node *head) {
    if (head == NULL) {
        cout << "Linked list kosong" << endl;
        return;
    }
    Node *temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

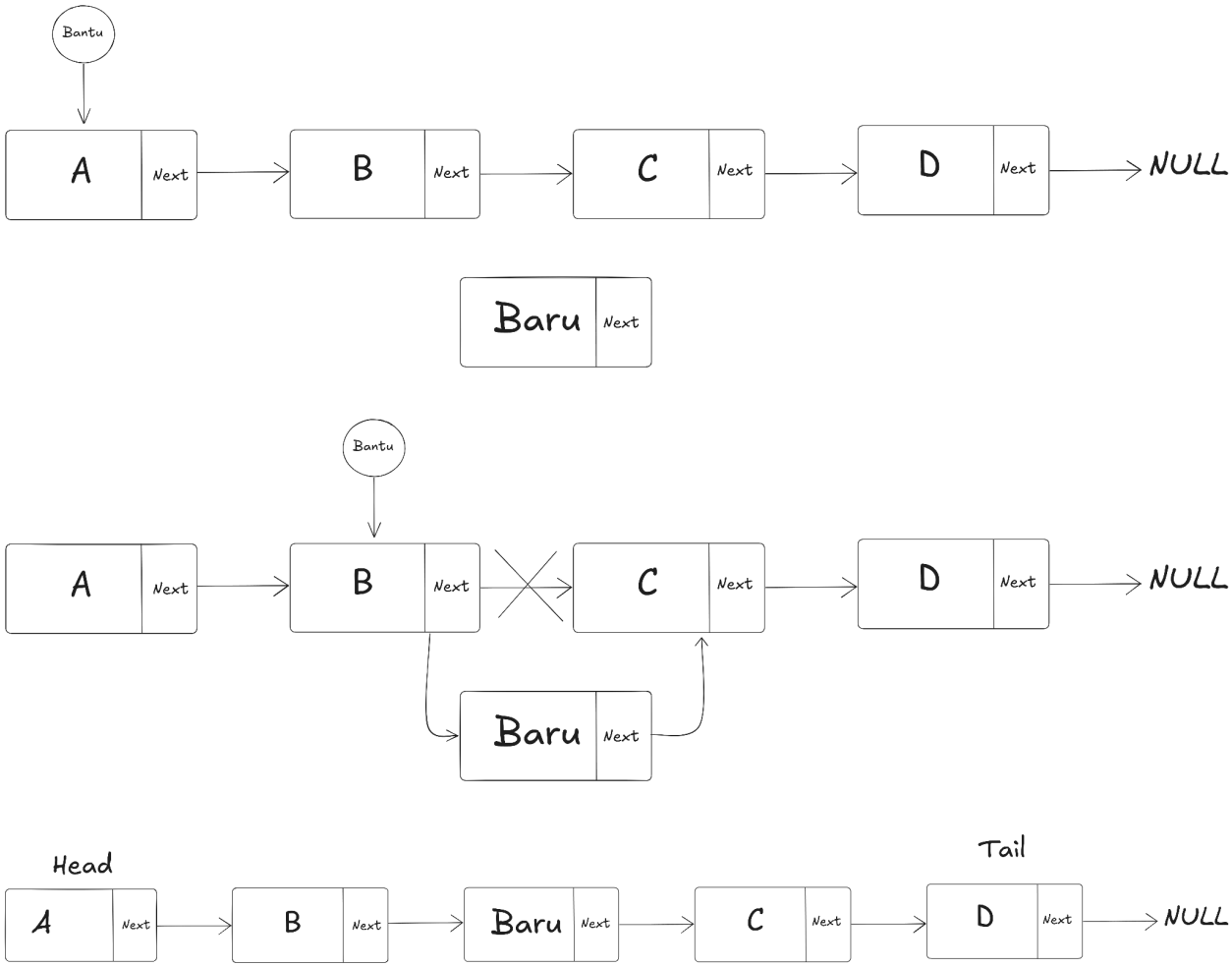
Add First



Add First merupakan penambahan data di bagian awal/depan linked list, sehingga menghasilkan head baru.

```
void addFirst(Node *&head, int databaru) {
    Node *nodeBaru = new Node;
    nodeBaru->data = databaru;
    nodeBaru->next = head;
    head = nodeBaru;
}
```

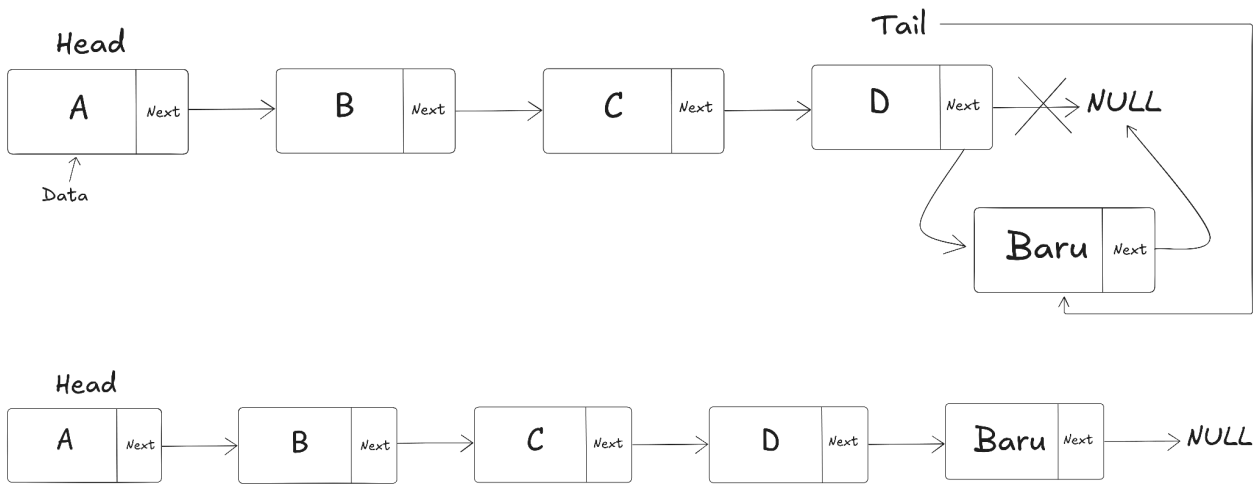
Add Specific



Add Specific merupakan penambahan data di antara head dan tail, biasanya user bisa memilih mau menghapus di urutan ke berapa.

```
void addMiddle(Node *&head, int databaru, int posisi) {
    if (posisi <= 1) {
        addFirst(head, databaru);
        return;
    }
    Node *temp = head;
    int hitung = 1;
    while (temp != nullptr && hitung < posisi - 1) {
        temp = temp->next;
        hitung++;
    }
    if (temp == nullptr) {
        cout << ">> Posisi tidak boleh melebihi panjang list <<" <<
endl;
        return;
    } else {
        Node *nodeBaru = new Node;
        nodeBaru->data = databaru;
        nodeBaru->next = temp->next;
        temp->next = nodeBaru;
    }
}
```

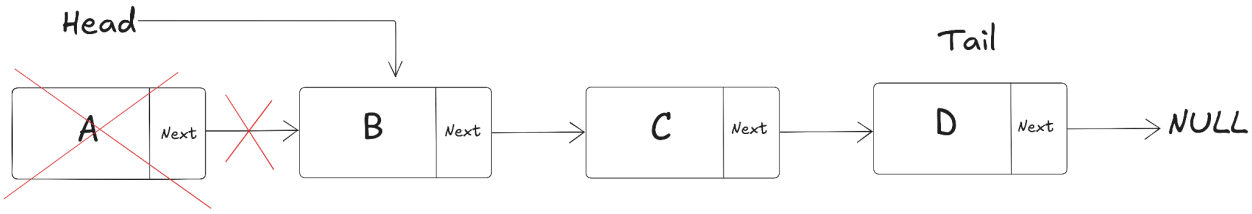
Add Last



Add Last merupakan penambahan data di bagian akhir/belakang linked list, sehingga menghasilkan tail baru.

```
void addLast(Node *&head, int databaru) {
    Node *nodeBaru = new Node;
    nodeBaru->data = databaru;
    nodeBaru->next = nullptr;
    // jika linked list kosong
    if (head == nullptr){
        head = nodeBaru;
        return;
    }
    // jika tidak, traversal ke node terakhir
    Node *temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = nodeBaru;
}
```

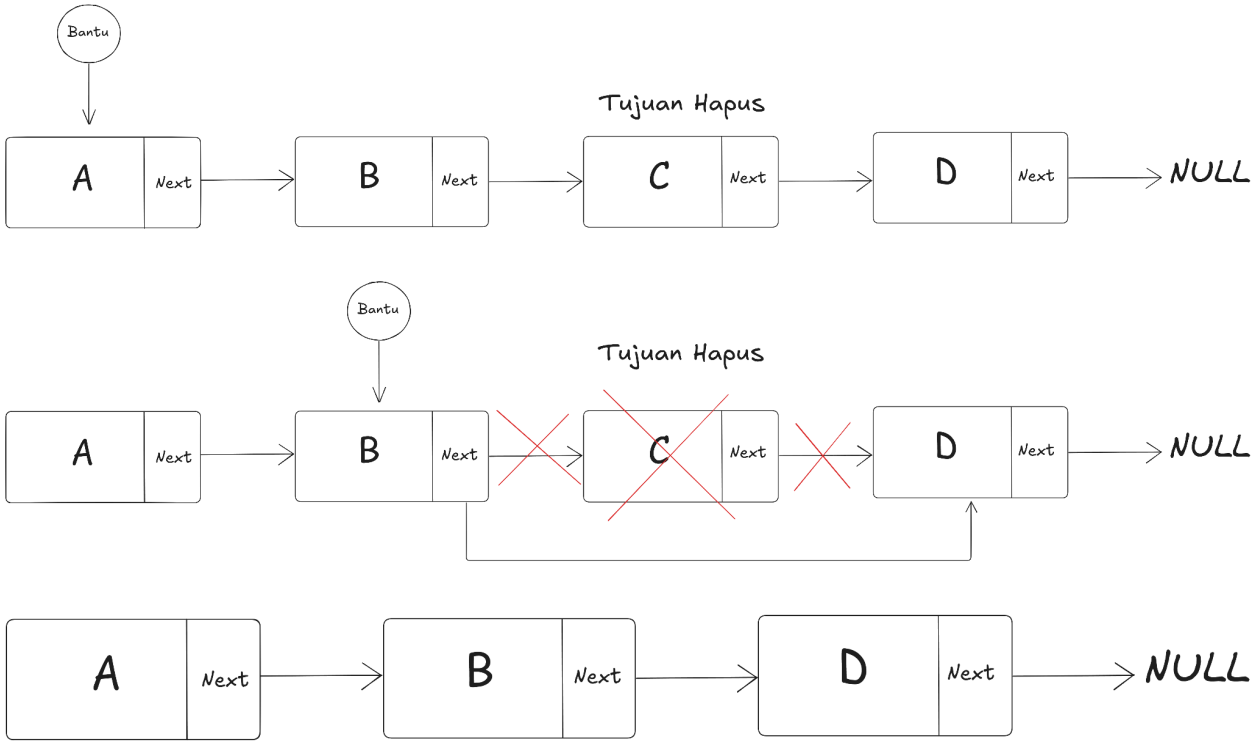
Delete First



Delete First merupakan penghapusan data di bagian awal/depan linked list.

```
void deleteFirst(Node *&head) {
    if (head == nullptr){
        cout << ">> LinkedList masih kosong <<" << endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    delete temp;
}
```

Delete Specific

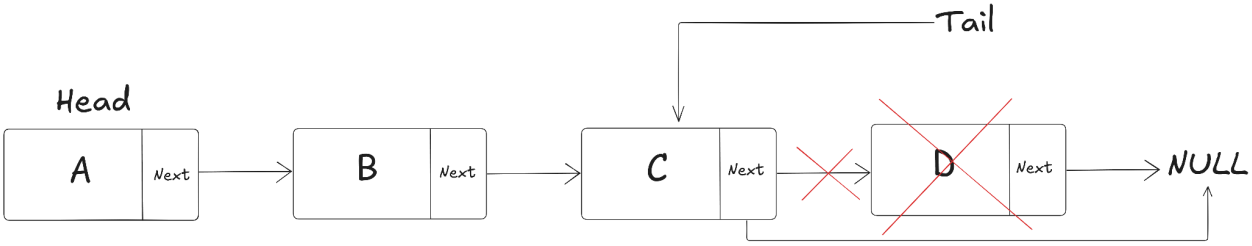


Delete Specific merupakan penghapusan data di antara head dan tail, biasanya user bisa memilih mau menghapus di urutan ke berapa

```
// Hapus data di tengah (posisi ke-n)
void deleteMiddle(Node *&head, int posisi) {
    if (head == nullptr) {
        cout << ">> LinkedList masih kosong <<" << endl;
        return;
    }
    if (posisi <= 1) {
        deleteFirst(head);
        return;
    }
    Node *temp = head;
    int hitung = 1;
    while (temp->next != nullptr && hitung < posisi - 1) {
```

```
        temp = temp->next;
        hitung++;
    }
    if (temp->next == nullptr) {
        cout << ">> Posisi tidak valid <<" << endl;
    } else {
        Node *hapus = temp->next;
        temp->next = hapus->next;
        delete hapus;
    }
}
```

Delete Last



Delete Last merupakan penghapusan data di bagian akhir/belakang linked list

```
void deleteLast(Node *&head) {
    if (head == nullptr){
        cout << ">> LinkedList masih kosong <<" << endl;
        return;
    }
        if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;
    }
    Node *temp = head;
    while (temp->next->next != nullptr){
        temp = temp->next;
```

```
    }
    delete temp->next;
    temp->next = nullptr;
}
```

Edit/Mengubah Data

```
void editData(Node *&head, int posisi, int dataBaru) {
    if (head == nullptr) {
        cout << ">> LinkedList masih kosong <<" << endl;
        return;
    }
    Node *temp = head;
    int hitung = 1;
    while (temp != nullptr && hitung < posisi) {
        temp = temp->next;
        hitung++;
    }
    if (temp == nullptr) {
        cout << ">> Posisi tidak valid <<" << endl;
    } else {
        cout << "Data lama: " << temp->data << endl;
        temp->data = dataBaru;
        cout << "Data berhasil diubah menjadi: " << temp->data << endl;
    }
}
```

Show/Menampilkan Data

```
void display(Node *head) {
    if (head == nullptr) {
        cout << ">> LinkedList kosong <<" << endl;
        return;
    }
    Node *temp = head;
    cout << "Isi LinkedList: ";
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```